

《创建自定义子代理 (Subagents) - Claude Code 文档》全文直译

页面标题：Create custom subagents - Claude Code Docs

原文链接： [Create custom subagents - Claude Code Docs](#)
[Source](#)

子代理是专门处理特定类型任务的专门 AI 助手。每个子代理都在其自己的上下文窗口中运行，具有自定义系统提示（system prompt）、特定工具访问权限以及独立的权限设置。当 Claude 遇到与子代理描述匹配的任务时，它会将任务委派给该子代理；该子代理独立工作并返回结果。子代理可以帮助你： [Source](#)

- **保留上下文**：将探索与实现从主对话中隔离出去
- **强约束**：限制子代理可以使用哪些工具
- **复用配置**：通过用户级子代理在项目之间复用配置
- **专门化行为**：为特定领域使用聚焦的系统提示
- **控制成本**：将任务路由到更快、更便宜的模型，例如 Haiku

[Source](#)

Claude 使用每个子代理的描述来决定何时委派任务。当你创建子代理时，写一个清晰的描述，以便 Claude 知道何时使用它。Claude Code 包含几个内置子代理，例如 **Explore**、**Plan** 和 **general-purpose**。你也可以创建自定义子代理来处理特定任务。本页涵盖：[内置子代理](#)、[如何创建你自己的子代理](#)、[完整配置选项](#)、[与子代理协作的模式](#) 以及 [示例子代理](#)。 [Source](#)

内置子代理（Built-in subagents）

Built-in subagents

Claude Code 包含内置子代理，当合适时 Claude 会自动使用它们。每个子代理都继承父对话的权限，并附加额外的工具限制。

Source

- Explore
- Plan
- General-purpose
- Other [Source](#)

Explore（内置） Explore

一个快速、只读的代理，针对搜索与分析代码库进行了优化。

Source

- **模型**：Haiku（快速、低延迟）
- **工具**：只读工具（拒绝访问 Write 与 Edit 工具）
- **目的**：文件发现、代码搜索、代码库探索 [Source](#)

当需要在不做更改的情况下搜索或理解代码库时，Claude 会委派给 Explore。这会把探索结果从主对话上下文中隔离出去。调用 Explore 时，Claude 会指定深入程度：**quick** 用于定向查找，**medium** 用于平衡探索，或 **very thorough** 用于全面分析。

Source

Plan（内置） Plan

一个研究代理，在 [计划模式（plan mode）](#) 期间使用，用于在呈现计划之前收集上下文。 [Source](#)

- **模型**：继承主对话
- **工具**：只读工具（拒绝访问 Write 与 Edit 工具）
- **目的**：为规划进行代码库研究 [Source](#)

当你处于计划模式并且 Claude 需要理解你的代码库时，它会把研究委派给 Plan 子代理。这可以防止无限嵌套（子代理不能生成其他子代理），同时仍能收集必要的上下文。 [Source](#)

General-purpose（内置） General-purpose

一个能力强的代理，用于需要探索与行动并存的复杂、多步骤任务。 [Source](#)

- **模型**：继承主对话
- **工具**：所有工具
- **目的**：复杂研究、多步骤操作、代码修改 [Source](#)

当任务需要探索与修改并存、需要复杂推理来解释结果，或需要多个相互依赖的步骤时，Claude 会委派给 general-purpose。

[Source](#)

Other（其他内置辅助代理） Other

Claude Code 还包含其他用于特定任务的辅助代理。它们通常会被自动调用，因此你不需要直接使用它们。 [Source](#)

代理	模型	Claude 何时使用它
Bash	继承	在单独上下文中运行终端命令
statusline-setup	Sonnet	当你运行 <code>/statusline</code> 配置状态栏时
Claude Code Guide	Haiku	当你询问 Claude Code 功能相关问题时
Source		

除这些内置子代理之外，你还可以使用自定义提示、工具限制、权限模式、钩子（hooks）和技能（skills）创建自己的子代理。以下部分展示如何开始并自定义子代理。 [Source](#)

快速开始：创建你的第一个子代理 (Quickstart) Quickstart

子代理在带有 YAML frontmatter 的 Markdown 文件中定义。你可以 [手动创建它们](#)，或使用 `/agents` 斜杠命令。本演练将指导你使用 `/agent` 命令创建一个用户级子代理。该子代理会审查代码并为代码库提出改进建议。 [Source](#)

注：抓取到的页面正文中，此处的 1–7 步的具体操作文本未完整呈现（只显示了数字占位）。下方为页面中可见的结果性描述。 [Source](#)

你现在有了一个子代理，可以在你机器上的任何项目中使用它来分析代码库并提出改进建议。你也可以手动以 Markdown 文件创建子代理、通过 CLI 标志定义它们，或通过插件分发它们。以下部分涵盖所有配置选项。 [Source](#)

配置子代理（Configure subagents） Configure subagents

使用 `/agents` 命令 Use the `/agents` command

`/agents` 命令提供一个用于管理子代理的交互式界面。运行

`/agents` 以： [Source](#)

- 查看所有可用子代理（内置、用户、项目、插件）
- 通过引导式设置或 Claude 生成来创建新子代理
- 编辑现有子代理的配置与工具访问权限
- 删除自定义子代理
- 查看当存在重名时，哪些子代理处于激活状态 [Source](#)

这是创建和管理子代理的推荐方式。对于手动创建或自动化，你也可以直接添加子代理文件。 [Source](#)

选择子代理的作用域（scope） Choose the subagent scope

子代理是带 YAML frontmatter 的 Markdown 文件。根据作用域将它们存放在不同位置。当多个子代理共享同一名称时，优先级更高的位置会生效。 [Source](#)

位置	作用域	优先级	如何创建
<code>--agents</code> CLI flag	当前会话	1 (最高)	启动 Claude Code 时传入 JSON
<code>.claude/agents/</code>	当前项目	2	交互式或手动
<code>~/.claude/agents/</code>	你的所有项目	3	交互式或手动
插件的 <code>agents/</code> 目录	插件启用处	4 (最低)	通过 <code>plugins</code> 安装
Source			

项目子代理（`.claude/agents/`）适用于特定代码库的子代理。将它们检入版本控制，以便团队协作使用与改进。**用户子代理**（`~/.claude/agents/`）是个人子代理，在你的所有项目中可用。**CLI 定义**的子代理在启动 Claude Code 时以 JSON 传入。它们仅存在于该会话且不会写入磁盘，因此适合快速测试或自动化脚本： **Source**

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use
proactively after code changes.",
    "prompt": "You are a senior code reviewer. Focus on
code quality, security, and best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  }
}'
```

Source

`--agents` 标志接受与 `frontmatter` 相同字段的 JSON。使用 `prompt` 作为系统提示（等同于基于文件的子代理中的 markdown 正文）。完整 JSON 格式参见 [CLI reference](#)。插件子代理来自你安装的 `plugins`，它们会与自定义子代理一起显示在 `/agents` 中。有关创建插件子代理的细节，参见 [plugin components reference](#)。 [Source](#)

编写子代理文件 Write subagent files

子代理文件使用 YAML frontmatter 进行配置，后面跟随 Markdown 格式的系统提示： [Source](#)

```
---
name: code-reviewer
description: Reviews code for quality and best practices
tools: Read, Glob, Grep
model: sonnet
---
```

You are a code reviewer. When invoked, analyze the code and provide specific, actionable feedback on quality, security, and best practices.

Source

frontmatter 定义子代理的元数据与配置。正文会成为引导子代理行为的系统提示。子代理只接收该系统提示（再加上工作目录等基础环境细节），而不是完整的 Claude Code 系统提示。 [Source](#)

支持的 frontmatter 字段 [Supported frontmatter fields](#)

以下字段可用于 YAML frontmatter。只有 `name` 和 `description` 是必需的。 [Source](#)

字段	必需	描述
<code>name</code>	是	使用小写字母和连字符的唯一标识符
<code>description</code>	是	Claude 何时应将任务委派给该子代理
<code>tools</code>	否	子代理可使用的 工具 。若省略则继承所有工具
<code>disallowedTools</code>	否	要禁止的工具，会从继承或指定的列表中移除
<code>model</code>	否	要使用的 模型 ： <code>sonnet</code> 、 <code>opus</code> 、 <code>haiku</code> 或 <code>inherit</code> 。默认 <code>sonnet</code>
<code>permissionMode</code>	否	权限模式 ： <code>default</code> 、 <code>acceptEdits</code> 、 <code>dontAsk</code> 、 <code>bypassPermissions</code> 或 <code>plan</code>
<code>skills</code>	否	启动时加载到子代理上下文中的 Skills 。注入的是完整技能内容，而不只是提供可调用入口。子代理不会从父对话继承技能
<code>hooks</code>	否	作用域限定于该子代理的 生命周期钩子
Source		

选择模型 Choose a model

`model` 字段控制子代理使用哪个 **AI 模型**：**Source**

- **模型别名**：使用可用别名之一：`sonnet`、`opus` 或 `haiku`
 - **inherit**：使用与主对话相同的模型（用于一致性）
 - **省略**：若未指定，则使用为子代理配置的默认模型（`sonnet`） [Source](#)
-

控制子代理能力（Control subagent capabilities） Control subagent capabilities

你可以通过工具访问、权限模式以及条件规则来控制子代理能做什么。 [Source](#)

可用工具 Available tools

子代理可以使用 Claude Code 的任何 **内部工具**。默认情况下，子代理会继承主对话的全部工具（包括 MCP 工具）。要限制工具，可使用 `tools` 字段（允许列表）或 `disallowedTools` 字段（拒绝列表）： [Source](#)

```
---
name: safe-researcher
description: Research agent with restricted capabilities
tools: Read, Grep, Glob, Bash
disallowedTools: Write, Edit
---
```

[Source](#)

权限模式 Permission modes

`permissionMode` 字段控制子代理如何处理权限提示。子代理继承主对话的权限上下文，但可以覆盖模式。 [Source](#)

模式	行为
<code>default</code>	标准权限检查并提示
<code>acceptEdits</code>	自动接受文件编辑
<code>dontAsk</code>	自动拒绝权限提示（明确允许的工具仍可工作）
<code>bypassPermissions</code>	跳过所有权限检查
<code>plan</code>	计划模式（只读探索）
Source	

如果父对话使用 `bypassPermissions`，它会优先生效且无法被覆盖。 [Source](#)

使用 hooks 的条件规则 Conditional rules with hooks

若需更动态地控制工具使用，可使用 `PreToolUse` hooks 在操作执行前进行校验。这在你需要允许某些工具的部分操作而阻止其它操作时很有用。下面示例创建一个仅允许只读数据库查询的子代理：`PreToolUse` hook 会在每次 Bash 命令执行前运行 `command` 中指定的脚本： [Source](#)

```

---
name: db-reader
description: Execute read-only database queries
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-
query.sh"
---

```

Source

Claude Code 会通过 stdin 以 JSON 形式传递 hook 输入（见 [pretooluse input](#)）。校验脚本读取该 JSON，提取 Bash 命令，并通过退出码 2 来阻止写操作： [Source](#)

```

#!/bin/bash
# ./scripts/validate-readonly-query.sh

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Block SQL write operations (case-insensitive)
if echo "$COMMAND" | grep -iE
'\b(INsert|UPDate|DELEte|DRoP|CREate|ALTER|TRUNCATE)\b'
> /dev/null; then
  echo "Blocked: Only SELECT queries are allowed" >&2
  exit 2
fi

exit 0

```

Source

完整输入 schema 见 [Hook input](#)，退出码行为见 [exit codes](#)。

Source

禁用特定子代理 Disable specific subagents

你可以通过在 [settings](#) 的 [deny](#) 数组中加入它们，来阻止 Claude 使用特定子代理。使用格式 `Task(subagent-name)`，其中

`subagent-name` 必须匹配子代理的 `name` 字段： [Source](#)

```
{
  "permissions": {
    "deny": [ "Task(Explore)", "Task(my-custom-agent)" ]
  }
}
```

Source

这对内置与自定义子代理都有效。你也可以使用 `--`

`disallowedTools` CLI 标志： [Source](#)

```
claude --disallowedTools "Task(Explore)"
```

Source

更多权限规则细节见 [IAM documentation](#)。 [Source](#)

为子代理定义 hooks（Define hooks for subagents） Define hooks for subagents

子代理可以定义在其生命周期中运行的 **hooks**。配置 hooks 有两种方式： **Source**

- 1. 在子代理的 frontmatter 中：定义仅在该子代理激活时运行的 hooks
- 2. 在 `settings.json` 中：定义在子代理启动或停止时在主会话中运行的 hooks **Source**

子代理 frontmatter 中的 hooks Hooks in subagent frontmatter

直接在子代理的 markdown 文件中定义 hooks。它们仅在该子代理激活期间运行，并在结束时清理。 **Source**

事件	matcher 输入	触发时机
<code>PreToolUse</code>	工具名	子代理使用工具之前
<code>PostToolUse</code>	工具名	子代理使用工具之后
<code>Stop</code>	(无)	子代理结束时
Source		

以下示例在 `PreToolUse` 中校验 Bash 命令，并在 `PostToolUse` 中对文件编辑后运行 linter： **Source**

```
---
name: code-reviewer
description: Review code changes with automatic linting
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
```

```
        command: "./scripts/validate-command.sh
$TOOL_INPUT"
    PostToolUse:
      - matcher: "Edit|Write"
        hooks:
          - type: command
            command: "./scripts/run-linter.sh"
---
```

Source

frontmatter 中的 `Stop` hooks 会自动转换为 `SubagentStop` 事件。 [Source](#)

项目级 hooks：响应子代理事件 Project-level hooks

在 `settings.json` 中配置 hooks，以便在主会话中响应子代理生命周期事件。使用 `matcher` 字段按名称定向特定代理类型。

Source

事件	matcher 输入	触发时机
<code>SubagentStart</code>	代理类型名	子代理开始执行时
<code>SubagentStop</code>	代理类型名	子代理执行完成时
Source		

示例：仅当 `db-agent` 子代理开始/停止时运行 setup 与 cleanup 脚本： [Source](#)

```
{
  "hooks": {
    "SubagentStart": [
      {
```

```
    "matcher": "db-agent",
    "hooks": [
      { "type": "command", "command":
"./scripts/setup-db-connection.sh" }
    ]
  },
],
"SubagentStop": [
  {
    "matcher": "db-agent",
    "hooks": [
      { "type": "command", "command":
"./scripts/cleanup-db-connection.sh" }
    ]
  }
]
}
```

Source

完整 hook 配置格式见 [Hooks](#)。 [Source](#)

使用子代理协作（Work with subagents） Work with subagents

理解自动委派 Understand automatic delegation

Claude 会基于你的请求中的任务描述、子代理配置中的 `description` 字段以及当前上下文自动委派任务。要鼓励更主动的委派，可在子代理的 `description` 字段中加入诸如 “use proactively” 的短语。你也可以显式请求某个子代理： [Source](#)

```
Use the test-runner subagent to fix failing tests
Have the code-reviewer subagent look at my recent
changes
```

[Source](#)

前台或后台运行子代理 Run subagents

子代理可以在前台（阻塞）或后台（并发）运行： [Source](#)

- **前台子代理**会阻塞主对话直到完成。权限提示与澄清问题（例如 `AskUserQuestion`）会透传给你。
- **后台子代理**会在你继续工作的同时并发运行。它们继承父级权限，并自动拒绝任何未预先批准的操作。如果后台子代理需要它没有的权限或需要提出澄清问题，则该工具调用会失败，但子代理会继续。后台子代理不可用 MCP 工具。 [Source](#)

如果后台子代理因缺少权限而失败，你可以在前台 **恢复它**，以使用交互式提示重试。Claude 会根据任务决定在前台还是后台运行子代理。你也可以： [Source](#)

- 请求 Claude “run this in the background”
- 按 **Ctrl+B** 将运行中的任务后台化 [Source](#)

要禁用所有后台任务功能，将环境变量

`CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` 设置为 `1`。参见 [Environment variables](#)。 [Source](#)

常见模式 (Common patterns)

Common patterns

隔离高输出量操作 Isolate high-volume operations

子代理最有效的用途之一，是隔离会产生大量输出的操作。运行测试、抓取文档或处理日志文件可能消耗大量上下文。通过把这些委派给子代理，冗长输出会留在子代理上下文里，而只有相关摘要返回到主对话。 [Source](#)

```
Use a subagent to run the test suite and report only the failing tests with their error messages
```

[Source](#)

并行研究 Run parallel research

对于彼此独立的调查，可以生成多个子代理同时工作： [Source](#)

```
Research the authentication, database, and API modules in parallel using separate subagents
```

[Source](#)

每个子代理会独立探索其领域，然后 Claude 综合这些发现。这在研究路径互不依赖时效果最好。运行许多子代理且每个都返回详细结果，会消耗大量上下文。 [Source](#)

串联子代理 Chain subagents

对于多步骤工作流，要求 Claude 按顺序使用子代理。每个子代理完成任务并返回结果给 Claude，Claude 再把相关上下文传给下一个子代理。 [Source](#)

```
Use the code-reviewer subagent to find performance
issues, then use the optimizer subagent to fix them
```

Source

在子代理与主对话之间选择 Choose between subagents and main conversation

在以下情况下使用主对话： [Source](#)

- 任务需要频繁来回沟通或迭代细化
- 多个阶段共享大量上下文（规划 → 实现 → 测试）
- 你在做一个快速、定向的更改
- 延迟很关键。子代理从空白开始，可能需要时间收集上下文

Source

在以下情况下使用子代理： [Source](#)

- 任务会产生你不希望进入主上下文的冗长输出
- 你希望强制特定工具限制或权限
- 工作是自包含的，可以返回摘要 [Source](#)

当你希望可复用的提示或工作流运行在主对话上下文中（而不是隔离的子代理上下文）时，考虑使用 [Skills](#)。 [Source](#)

子代理不能生成其他子代理。如果你的工作流需要嵌套式委派，请使用 [Skills](#) 或在主对话中 [串联子代理](#)。 [Source](#)

管理子代理上下文 (Manage subagent context) Manage subagent context

恢复子代理 Resume subagents

每次调用子代理都会创建一个新的实例，并拥有全新的上下文。要继续一个既有子代理的工作而不是从头开始，请让 Claude 恢复它。被恢复的子代理保留完整对话历史，包括之前所有工具调用、结果与推理。子代理会从停止处继续，而不是重新开始。子代理完成时，Claude 会收到其 agent ID。要恢复子代理，可请求 Claude 继续之前的工作： [Source](#)

```
Use the code-reviewer subagent to review the authentication module
```

```
[Agent completes]
```

```
Continue that code review and now analyze the authorization logic
```

```
[Claude resumes the subagent with full context from previous conversation]
```

Source

你也可以询问 Claude agent ID 以便显式引用，或在

```
~/.claude/projects/{project}/{sessionId}/subagents/
```

的转录文件中找到 ID。每个转录存储为 `agent-`

`{agentId}.jsonl`。程序化使用参见 [Subagents in the Agent](#)

[SDK](#)。子代理转录会独立于主对话持久化： [Source](#)

- **主对话压缩 (compaction)**：主对话压缩时，子代理转录不受影响；它们存储在单独文件中。

- **会话持久化**：子代理转录在其会话内持久化。重启 Claude Code 后，你可通过恢复同一会话来 [恢复子代理](#)。
- **自动清理**：转录会根据 `cleanupPeriodDays` 设置清理（默认：30 天）。 [Source](#)

自动压缩（Auto-compaction） Auto-compaction

子代理支持与主对话相同逻辑的自动压缩。默认情况下，自动压缩在约 95% 容量时触发。要更早触发压缩，将

`CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` 设置为更低百分比（例如 50）。参见 [environment variables](#)。压缩事件会记录在子代理转录文件中： [Source](#)

```
{
  "type": "system",
  "subtype": "compact_boundary",
  "compactMetadata": {
    "trigger": "auto",
    "preTokens": 167189
  }
}
```

[Source](#)

`preTokens` 值表示在发生压缩前使用了多少 token。 [Source](#)

示例子代理（Example subagents） Example subagents

这些示例展示了构建子代理的有效模式。将它们作为起点，或让 Claude 生成定制版本。最佳实践：设计聚焦的子代理（每个子代理擅长一件事）、编写详细描述（Claude 依据描述决定何时委派）、限制工具访问（为安全与聚焦仅授予必要权限）、检入版本控制（与团队共享项目子代理）。 [Source](#)

代码审查员（Code reviewer） Code reviewer

一个只读子代理，用于在不修改代码的情况下审查代码。该示例展示了如何设计一个聚焦的子代理、限制工具访问（不含 Edit 或 Write），并用详细提示说明检查点与输出格式。 [Source](#)

```
---
name: code-reviewer
description: Expert code review specialist. Proactively
reviews code for quality, security, and maintainability.
Use immediately after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
---
```

You are a senior code reviewer ensuring high standards of code quality and security.

When invoked:

1. Run `git diff` to see recent changes
2. Focus on modified files
3. Begin review immediately

Review checklist:

- Code is clear and readable
- Functions and variables are well-named
- No duplicated code
- Proper error handling
- No exposed secrets or API keys

- Input validation implemented
- Good test coverage
- Performance considerations addressed

Provide feedback organized by priority:

- Critical issues (must fix)
- Warnings (should fix)
- Suggestions (consider improving)

Include specific examples of how to fix issues.

Source

调试器 (Debugger) Debugger

一个既能分析也能修复问题的子代理。与代码审查员不同，它包含 Edit，因为修复 bug 需要修改代码。提示给出了从诊断到验证的清晰工作流。 [Source](#)

```
---
name: debugger
description: Debugging specialist for errors, test
failures, and unexpected behavior. Use proactively when
encountering any issues.
tools: Read, Edit, Bash, Grep, Glob
---
```

You are an expert debugger specializing in root cause analysis.

When invoked:

1. Capture error message and stack trace
2. Identify reproduction steps
3. Isolate the failure location
4. Implement minimal fix

5. Verify solution works

Debugging process:

- Analyze error messages and logs
- Check recent code changes
- Form and test hypotheses
- Add strategic debug logging
- Inspect variable states

For each issue, provide:

- Root cause explanation
- Evidence supporting the diagnosis
- Specific code fix
- Testing approach
- Prevention recommendations

Focus on fixing the underlying issue, not the symptoms.

Source

数据科学家（Data scientist） Data scientist

一个用于数据分析工作的领域子代理。该示例展示如何为典型编码任务之外的专门工作流程创建子代理，并显式设置 `model: sonnet` 以获得更强分析能力。 [Source](#)

```
---
name: data-scientist
description: Data analysis expert for SQL queries,
BigQuery operations, and data insights. Use proactively
for data analysis tasks and queries.
tools: Bash, Read, Write
model: sonnet
---
```


You are a data scientist specializing in SQL and BigQuery analysis.

When invoked:

1. Understand the data analysis requirement
2. Write efficient SQL queries
3. Use BigQuery command line tools (bq) when appropriate
4. Analyze and summarize results
5. Present findings clearly

Key practices:

- Write optimized SQL queries with proper filters
- Use appropriate aggregations and joins
- Include comments explaining complex logic
- Format results for readability
- Provide data-driven recommendations

For each analysis:

- Explain the query approach
- Document any assumptions
- Highlight key findings
- Suggest next steps based on data

Always ensure queries are efficient and cost-effective.

Source

数据库查询校验器（Database query validator / db-reader） Database query validator

一个允许 Bash 访问但会校验命令、仅允许只读 SQL 查询的子代理。该示例展示当你需要比 `tools` 字段更细粒度控制时，如何用 `PreToolUse` hooks 做条件校验。 [Source](#)

```
name: db-reader
description: Execute read-only database queries. Use
when analyzing data or generating reports.
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-
query.sh"
---
```

You are a database analyst with read-only access.
Execute SELECT queries to answer questions about the
data.

When asked to analyze data:

1. Identify which tables contain the relevant data
2. Write efficient SELECT queries with appropriate filters
3. Present results clearly with context

You cannot modify data. If asked to INSERT, UPDATE, DELETE, or modify schema, explain that you only have read access.

Source

Claude Code 通过 stdin 将 hook 输入以 JSON 形式传递给 hook 命令。校验脚本读取 JSON，提取正在执行的命令，并与 SQL 写操作列表比对；检测到写操作时，脚本以退出码 2 阻止执行并通过 stderr 返回错误信息。将校验脚本放在项目中的任意位置，路径必须与 hook 配置中的 `command` 字段匹配： [Source](#)

```
#!/bin/bash

# Blocks SQL write operations, allows SELECT queries

# Read JSON input from stdin
INPUT=$(cat)

# Extract the command field from tool_input using jq
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

if [ -z "$COMMAND" ]; then
    exit 0
fi

# Block write operations (case-insensitive)
if echo "$COMMAND" | grep -iE
'\b(INsert|UPdate|DElete|DRop|CREate|ALter|TRUnCate|REPL
ACE|MERGE)\b' > /dev/null; then
    echo "Blocked: Write operations not allowed. Use
SELECT queries only." >&2
    exit 2
fi

exit 0
```

Source

使脚本可执行: [Source](#)

```
chmod +x ./scripts/validate-readonly-query.sh
```

Source

该 hook 通过 stdin 接收 JSON，其中 Bash 命令位于 `tool_input.command`。退出码 2 会阻止操作并把错误信息回传给 Claude。退出码细节见 [Hooks exit-codes](#)，输入 schema 见 [Hook input](#)。 [Source](#)

下一步（Next steps） Next steps

现在你已经理解了子代理，可以继续探索这些相关功能： [Source](#)

- [通过 plugins 分发子代理](#)，以跨团队或跨项目共享子代理
- [以编程方式运行 Claude Code](#)，使用 Agent SDK 进行 CI/CD 与自动化
- [使用 MCP servers](#)，为子代理提供对外部工具与数据的访问 [Source](#)